Los Sabrossos

Contents

1 STRINGS

1.1 Hashing

```
struct Hash {
  long long P=1777771,MOD[2],PI[2];
  vector<long long> h[2],pi[2];
  Hash(string& s){
    MOD[0]=999727999;MOD[1]=1070777777;
    PI[0]=325255434;PI[1]=10018302;
    for(int k = 0; k < 2; k++) h[k].resize(s.size()+1),pi[k].resize(s.size()+1);
    for(int k = 0; k < 2; k++){
      h[k][0]=0;pi[k][0]=1;
      long long p=1;
            for(int i = 1; i < s.size()+1; i++){
        h[k][i]=(h[k][i-1]+p*s[i-1])%MOD[k];
        pi[k][i]=(1LL*pi[k][i-1]*PI[k])%MOD[k];
        p=(p*P)%MOD[k];
      }
    }
  }
  long long get(int s, int e){
    long long h0=(h[0][e]-h[0][s]+MOD[0])%MOD[0];
    h0=(1LL*h0*pi[0][s])%MOD[0];
    long long h1=(h[1][e]-h[1][s]+MOD[1])%MOD[1];
    h1=(1LL*h1*pi[1][s])%MOD[1];
    return (h0<<32)|h1;
  }
};
```

1.2 Kmp

```
vector<int> kmp(string s){
    int n = s.size();
    vector<int> pi(n);
    for(int i = 1; i < n; i++){
        int j = pi[i-1];
        while(j > 0 and s[i] != s[j]){
            j = pi[j-1];
        }
        if(s[i] == s[j]){
           j++;
        }
        pi[i] = j;
    }
    return pi;
}
//Pattern Matching
vector<int> ocurrencia(string texto, string patron) {
    vector<int> P = kmp(patron);
    int k = 0;
    vector<int> M;
    repl(i,0,texto.size()) {
        while (k > 0 and patron[k] != texto[i]) k = P[k - 1];
        if (patron[k] == texto[i]) k++;
        if (k == patron.size()) {
            M.pb(i - k + 1);
            k = P[k - 1];
        }
    }
    return M;
```

```
}

//Automata
void genAut(string &s) {
    int n = s.size();

    vector<vector<int>> aut(n+1, vector<int>(26));
    vector<int> pi = kmp(s);

    for(int i = 0 ; i < n; i++){
        aut[i][s[i]-'a'] = i+1;
    }

    for(int i = 0; i < n+1; i++){
        for(int c = 0; c < 26; c++){
            if (aut[i][c]) continue;
            if (i < n and s[i] == 'a' + c) aut[i][c] = i+1;
            else if (i > 0) aut[i][c] = aut[pi[i-1]][c];
            else aut[i][c] = 0;
        }
    }
}
```

2 TREE ALGORITHMS

2.1 Binary lifting

```
const int N = 1e5;
const int LOG = 20;
int up[N][LOG];
vector<int> G[N];
//Find parents
void dfs(int u){
    for(int v : G[u]){
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_K_parent(int a, int k){
    for(int j = LOG-1; j >= 0; j++){
        if(k&(1<<j)){
            a = up[a][j];
        }
    }
    return a;
}
```

2.2 Lowest common ancestor

```
const int N = 1e5;
const int LOG = 20;
vector<int> G[N];
int depth[N];
int up[N][LOG];
// Assign levels and get parents
void dfs(int u){
```

```
    for(int v : G[u]){
        depth[v] = depth[u] + 1;
        up[v][0] = u;
        for(int j = 1; j < LOG; j++){
            up[v][j] = up[up[v][j-1]][j-1];
        }
        dfs(v);
    }
}
int get_lca(int a, int b){
    if(depth[a] < depth[b]) swap(a,b);
    int k = depth[a] - depth[b];
    for(int j = LOG-1; j >= 0; j--){
        if(k & (1<<j)){
            a = up[a][j];
        }
    }
    if(a == b) return a;
    for(int j = LOG-1; j >= 0; j--){
        if(up[a][j] != up[b][j]){
            a = up[a][j];
            b = up[b][j];
        }
    }
    return up[a][0];
}
```

3 DATA STRUCTURES

3.1 Disjoint set union

```
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
```

```
        }
        else{
            parent[y] = x;
            sizes[x]+=sizes[y];
        }

    }
};
```

3.2 Segment tree

```
const int N = 1e5;
struct info{
    int num;
};
info ST[4*N];
info ope(info u, info v){
    return {u.num+v.num};
}
void build(int in, int L, int R){
    if(L == R){
        cin >> ST[in].num;
    }
    else{
        int mid = (L+R)>>1;
        build((2*in),L,mid);
        build((2*in)+1,mid+1,R);
        ST[in] = ope(ST[(2*in)], ST[(2*in)+1]);
    }
}
void update(int in, int L, int R, int pos, int val){
    if(L == R){
        ST[in].num = val;
    }
    else{
        int mid = (L+R)>>1;
        if(pos <= mid){
            update(2*in,L,mid,pos,val);
        }
        else{
            update((2*in)+1,mid+1,R,pos,val);
        }
        ST[in] = ope(ST[2*in], ST[(2*in)+1]);
    }
}
info get(int in, int L, int R, int l, int r){
    if(r < L or R < l) return {0};
    if(l <= L and R <= r) return ST[in];
    int mid = (L+R)>>1;
    info left = get(2*in,L,mid,l,r);
    info right = get((2*in)+1,mid+1,R,l,r);
    return ope(left,right);
}
// v = {1 2 3 4 5}
// get(1, 0, n-1, 1, 3) = 2 + 3 + 4 = 9
```

3.3 Sparse table

```
const int N = 1e5;
const int LOG = 20;
```

```cpp
int v[N]; //values
int st[N][LOG]; //Sparse Table
int lg[N]; // log values
int n;
void build(){
    for(int i = 0; i < n; i++) st[i][0] = v[i];

    for(int i = 1; i < LOG; i++){
        for(int j = 0; j < n - (1<<i) + 1; j++){
            st[j][i] = min(st[j][i-1],st[j+(1<<(i-1))][i-1]);
        }
    }
    lg[1] = 0;
    for(int i = 2; i <= n;i++) lg[i] = lg[i/2] + 1;
}
int query(int L, int R){
    int k = lg[R-L+1];
    return min(st[L][k],st[R-(1<<k)+1][k]);
}
// v = [5, 2, 4, 7, 1, 3]
// query(1, 4) → [2, 4, 7, 1]
```

4 MATH

4.1 Binary exponentiation

```cpp
const long long mod = 998244353;
long long pote(long long a, long long b){
    long long ans = 1ll;
    while(b){
        if(b&1) ans = (ans * a)%mod;
        b>>=1;
        a = (a * a)%mod;
    }
    return ans;
}
```

4.2 Modular inverse

```cpp
const long long N = 1e6 + 10;
const int MOD = 1e9+7;
long long fact[N];
long long invfact[N];
void init(){
    fact[0] = 1;
    for(long long i = 1; i < N; i++)
        fact[i] = fact[i-1] * i % MOD;

    invfact[N-1] = pote(fact[N-1], MOD-2);

  for(long long i = N-2; i >= 0; i--)
        invfact[i] = invfact[i+1] * (i+1) % MOD;
}

long long nCk(long long n, long long k){
    if(k < 0 || k > n) return 0;
    return fact[n] * invfact[k] % MOD * invfact[n-k] % MOD;
}
```

4.3 Sieve of eratosthenes

```
const int N = 1e7;
int sieve[N];
void make(){
    for(int i = 0; i < N; i++) sieve[i] = 1;
    sieve[0] = sieve[1] = 0;
    for (int i = 2; i < N; i++)
    {
        if(sieve[i]){
            for (int j = i*i; j < N; j+=i)
            {
                sieve[j] = 0;
            }
        }
    }
}
```

5 GRAPH ALGORITHMS

5.1 Bfs

```
const int N = 1e5;

vector<int> G[N];
int vis[N];
void bfs(int source){
    queue<int> q;
    q.push(source);
    vis[source] = 0;
    while(!q.empty()){
        int u = q.front();
        q.pop();
        for(int v : G[u]){
            if(vis[v] == -1){
                vis[v] = vis[u] + 1;
                q.push(v);
            }
        }
    }
}
```

5.2 Dfs

```
const int N = 1e5;

vector<int> G[N];
int vis[N];

void dfs(int u){
    vis[u] = 1;
    for(int v : G[u]){
        if(vis[v] == -1){
            dfs(v);
        }
    }
}
```

5.3 Dijkstra

```cpp
const int N = 1e5;

int vis[N];
vector<pair<int,int>> G[N];

void dijkstra(int x){
    priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;
    pq.push({0,x});
    vis[x] = 0;
    while(!pq.empty()){
        auto y = pq.top();
        pq.pop();
        int u = y.second;
        int cost = y.first;
        for(auto v : G[u]){
            int cur = cost + v.second;
            if(cur < vis[v.first]){
                vis[v.first] = cur;
                pq.push({cur,v.first});
            }
        }
    }
}
```

5.4 Kruskal mst

```cpp
const int N = 1e5;
struct DSU{
    int num;
    vector<int> parent;
    vector<int> sizes;
    DSU(){};
    DSU(int n){
        init(n);
    }
    void init(int n){
        num = n;
        parent.resize(n+1);
        sizes.resize(n+1);
        for(int i = 1; i <= n ; i++){
            parent[i] = i;
            sizes[i] = 1;
        }
    }
    int find(int a){
        if(a == parent[a]) return a;
        return parent[a] = find(parent[a]);
    }
    void join(int a, int b){
        int x = find(a);
        int y = find(b);
        if(x == y) return;
        num--;
        if(sizes[x] < sizes[y]){
            parent[x] = y;
            sizes[y]+=sizes[x];
        }
        else{
            parent[y] = x;
```

```
            sizes[x]+=sizes[y];
        }
    }
};
int n;
vector<pair<int,pair<int,int>>> List;
int Kruskal_MST(){
    sort(List.begin(),List.end());
    DSU dsu(n);
    int mst = 0, t = 1;
    for(int i = 0; i < List.size(); i++){
        int w = List[i].first; //weigth
        int a = List[i].second.first; //at
        int b = List[i].second.second; //to
        if(dsu.find(a) != dsu.find(b)){
            dsu.join(a,b);
            mst+=w;
            t++;
        }
        if(t == n) break;
    }
    return mst;
}
```

5.5 Topological sort kahn algorithm

```
const int N = 1e5;

int indegree[N];
vector<int> G[N];
int n;
vector<int> Kahn(){
    queue<int> q;
    vector<int> ans;
    for(int i = 0; i < n; i++){
        if(indegree[i] == 0) q.push(i);
    }
    while (!q.empty()){
        int v = q.front();
        q.pop();
        ans.push_back(v);
        for(int u : G[v]){
            indegree[u]--;
            if(indegree[u] == 0){
                q.push(u);
            }
        }
    }
    //if(ans.size() != n) grafo no DAG
    return ans;
}
```